

PROJETO FAZENDA URBANA

Autor: Lucas Natal

N1 DO PROJETO

1 INTRODUÇÃO

Esse projeto consiste no desenvolvimento de um site denominado “Fazenda Urbana”, com o objetivo de apresentar uma solução sustentável para problemas urbanos relacionados à alimentação, desperdício de recursos e uso ineficiente de espaços.

A proposta do projeto é demonstrar como a tecnologia pode ser utilizada para promover impacto social positivo, utilizando conceitos de desenvolvimento web aliados a temas de sustentabilidade.

O projeto foi desenvolvido com base nos (ODS), abordando principalmente:

- **ODS 2 – Fome Zero e Agricultura Sustentável**
O projeto propõe soluções para melhorar o acesso a alimentos frescos e reduzir a insegurança alimentar.
- **ODS 11 – Cidades e Comunidades Sustentáveis**
Utiliza espaços urbanos disponíveis para produção de alimentos, promovendo sustentabilidade nas cidades.
- **ODS 12 – Consumo e Produção Responsáveis**
Enfatiza a redução do desperdício e o uso eficiente de recursos.
- **ODS 4 – Educação de Qualidade**
Incentiva a educação ambiental e a conscientização da população.

2.1 ORGANIZAÇÃO DOS ARQUIVOS

A estrutura do projeto foi organizada visando facilitar a manutenção e escalabilidade. A separação foi feita da seguinte maneira:

- **Index.html:** página principal.
- **Html (páginas):** responsáveis pela estrutura do site.
- **CSS:** responsável pela estilização e layout.
- **JavaScript:** responsável pela interatividade.
- **Assets (imagens):** armazenamento de imagens.

2.2 ESTRUTURA DAS PÁGINAS

- **Home (index):** apresenta o projeto e contém o carrossel de imagens.

- **Etapas:** descreve o processo de desenvolvimento da solução.
- **Impactos:** apresenta os impactos sociais e ambientais.
- **Parceiros:** lista instituições e empresas envolvidas.
- **Contato:** permite interação com o usuário por meio de formulário.

2.3 LISTAR AS ROTAS

Foram definidos dois tipos principais de rotas:

- **GET:** utilizado para navegação entre páginas.
- **POST:** utilizado para envio de dados do formulário.

Rota	Método	Descrição	Página
/	GET	Página principal.	index.html
/etapas	GET	Etapas do projeto.	etapas.html
/impactos	GET	Impactos sociais.	impactos.html
/parceiros	GET	Página de parceiros.	parceiros.html
/contato	GET	Exibe o formulário.	contato.html
/contato	POST	Envia os dados do formulário ao servidor.	contato.html

2.5 ÁRVORE DE ESTRUTURA

FazendaUrbana/

|----- index.html

|----- css/

| |----- style.css

|-----js/

| |-----imagem.js

| |-----contato.js

|-----assets/

| |-----img/

|-----pages/

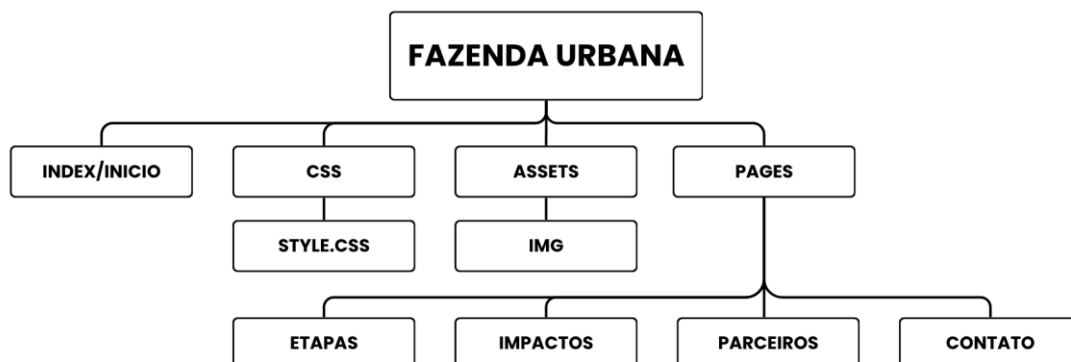
| |-----etapas.html

| |-----impactos.html

| |----- parceiros.html

| |-----contato.html

2.6 VISUALIZAÇÃO EM IMAGEM



3.1 JAVASCRIPT

O JavaScript foi utilizado para adicionar interatividade ao projeto, sendo aplicado em duas funcionalidades principais:

3.2 CARROSSEL DE IMAGENS

- Navegação por botões (avancar e voltar)
- Manipulação do estilo transform para movimentação horizontal
- Uso de translateX para trocar as imagens
- Transições suaves utilizando CSS

3.3 VALIDAÇÃO DE FORMULÁRIO

Foi implementada a validação de dados, verificando:

- Se o nome foi preenchido.
- Se o e-mail foi preenchido.
- Se o número foi preenchido.

Caso algum campo esteja vazio, o sistema exibe mensagens de alerta, após isso os campos são limpos automaticamente.

4 DECISÕES TÉCNICAS

Durante o desenvolvimento do projeto, foram tomadas algumas decisões importantes com o objetivo de garantir organização e facilidade de manutenção do código. Separou os arquivos em diferentes pastas, dividindo HTML, CSS, JavaScript e assets, o que facilita a estruturação. Para o layout, foi utilizado o Flexbox,

permitindo um melhor alinhamento dos elementos e contribuindo para a responsividade da aplicação em diferentes tamanhos de tela. Também foram utilizadas variáveis no CSS, com o intuito de padronizar cores, espaçamentos e outros elementos visuais, tornando o código mais reutilizável e consistente.

O uso do JavaScript foi aplicado na interatividade do carrossel de imagens e na validação do formulário. Além disso, o sistema foi estruturado desde o início pensando em uma futura integração com back-end, especialmente na configuração do formulário com método POST, o que permite a expansão do projeto para incluir persistência de dados em etapas posteriores.

5 CONCLUSÃO

O desenvolvimento deste projeto permitiu a aplicação prática de conceitos fundamentais de desenvolvimento web, além de promover reflexão sobre sustentabilidade e impacto social.

A estrutura criada serve como base para futuras melhorias, como integração com back-end, armazenamento de dados e expansão das funcionalidades.

6 RECURSOS E ATIVOS

Imagens: Todas as imagens localizadas em `/assets/img/` foram geradas via Inteligência Artificial para fins exclusivamente acadêmicos, visando evitar o uso de conteúdo protegido por direitos autorais.

N2 DO PROJETO - EXPANSÃO DO PROJETO

7 IMPLEMENTAÇÃO BACK-END

Nesta segunda etapa do projeto, o sistema Fazenda Urbana foi expandido para uma aplicação web full stack, integrando front-end, back-end e banco de dados. O objetivo desta etapa foi transformar o site anteriormente estático em um sistema funcional, com persistência de dados e rotas dinâmicas.

Foram utilizadas tecnologias como Node.js, Express.js, MongoDB, Mongoose, EJS e Fetch API.

7.1 NODE.JS E EXPRESS

O Node.js foi utilizado para execução do JavaScript ao lado do servidor, permitindo criação do back-end da aplicação. E o Express.js foi utilizado para simplificar a criação das rotas HTTP e gerenciamento do servidor.

No arquivo server.js foi implementado diferentes rotas HTTP seguindo o padrão CRUD.

Método	Função
GET	Navegação entre páginas
POST	Cadastro, login e envio de formulários.
PUT	Atualização de contatos.
DELETE	Remoção de usuários.

METODO GET - RENDERIZAR AS PÁGINAS DO SISTEMA UTILIZANDO EJS.

```
137  /* ROTAS GET */
138
139  app.get("/", (req, res) => res.render("index"));
140  app.get("/login", (req, res) => res.render("login", { query: req.query }));
141  app.get("/etapas", (req, res) => res.render("etapas"));
142  app.get("/impactos", (req, res) => res.render("impactos"));
143  app.get("/parceiros", (req, res) => res.render("parceiros"));
144  app.get("/contato", (req, res) => res.render("contato", { query: req.query }));
145
146  app.get("/api/status", (req, res) => {
147    res.json({
148      mongo: bancoConectado(),
149      banco: "fazendaurbana",
150      colecoes: ["users", "contatos"]
151    });
152  });
```

rotas são responsáveis por carregar as páginas do sistema no navegador.

POST - CRIAÇÃO E ENVIO DE DADOS.

```

27  /* ROTAS POST */
28
29  app.post("/registro", async (req, res) => {
30    try {
31      if (!bancoConectado()) {
32        if (pediuJson(req)) {
33          return res.status(503).json({ sucesso: false, mensagem: "MongoDB desligado. Inicie o MongoDB." });
34        }
35        return res.redirect("/login?erro=mongo");
36      }
37
38      const nome = (req.body.nome || "").trim();
39      const email = (req.body.email || "").trim();
40      const usuario = (req.body.usuario || "").trim();
41      const senha = req.body.senha || "";
42
43      if (!nome || !email || !usuario || !senha) {
44        if (pediuJson(req)) {
45          return res.json({ sucesso: false, mensagem: "Preencha todos os campos." });
46        }
47        return res.redirect("/login?erro=campos");
48      }
49
50      const jaExiste = await User.findOne({ usuario: usuario });
51      if (jaExiste) {
52        if (pediuJson(req)) {
53          return res.json({ sucesso: false, mensagem: "Usuário já existe." });
54        }
55        return res.redirect("/login?erro=usuario");
56      }
57
58      await User.create({ nome, email, usuario, senha });
59      console.log("Usuário Criado:", usuario);
60
61      if (pediuJson(req)) {
62        return res.json({ sucesso: true, mensagem: "Conta criada!" });
63      }
64      return res.redirect("/login?cadastro=ok");
65    } catch (erro) {
66      console.log("Erro registro:", erro.message);
67      if (pediuJson(req)) {
68        return res.json({ sucesso: false, mensagem: "Erro ao criar conta." });
69      }
70      return res.redirect("/login?erro=servidor");
71    }
72  });

```

O método POST foi utilizado para cadastrar usuários, realizar login e enviar formulários de contato. Esse é um exemplo para o registro de novos usuários.

PUT - ATUALIZAÇÃO DE DADOS.

```

156  /* ROTAS PUT */
157
158  app.put("/contato/:id", async (req, res) => {
159    try {
160
161      await Contato.findByIdAndUpdate(
162        req.params.id,
163        {
164          nome: req.body.nome,
165          email: req.body.email,
166          numero: req.body.numero
167        }
168      );
169
170      res.json({
171        sucesso: true,
172        mensagem: "Contato atualizado!"
173      });
174
175    } catch (erro) {
176
177      res.status(500).json({
178        sucesso: false,
179        mensagem: "Erro ao atualizar contato."
180      });
181
182    }
183  });

```

O método PUT foi utilizado para atualizar informações já existentes no banco de dados. (Não tem interações no site, criei o PUT e o DELETE por que o projeto exigia.)

DELETE - REMOÇÃO DE DADOS


```

185  /* ROTAS DELETE */
186
187  app.delete("/usuario/:id", async (req, res) => {
188    try {
189
190      await User.findByIdAndDelete(req.params.id);
191
192      res.json({
193        sucesso: true,
194        mensagem: "Usuário removido!"
195      });
196
197    } catch (erro) {
198
199      res.status(500).json({
200        sucesso: false,
201        mensagem: "Erro ao remover usuário."
202      });
203
204    }
205  });

```

O método DELETE foi utilizado para remover registros do banco de dados.

7.2 MONGODB E MONGOOSE

O MongoDB foi responsável pelo armazenamento das informações dos usuários cadastrados e das solicitações enviadas pelo formulário de contato. A conexão com o banco de dados está no arquivo connection.js.

```

3  const mongoose = require("mongoose");
4
5  async function conectarBanco() {
6    try {
7      await mongoose.connect("mongodb://127.0.0.1:27017/fazendaurbana");
8      console.log("MongoDB conectado!");
9      return true;
10   } catch (erro) {
11     console.log("Erro MongoDB:", erro.message);
12     return false;
13   }
14 }
15
16 module.exports = conectarBanco;

```

Junto foi utilizado para criar Schemas para estruturar os dados de contato e de login, que está na pasta Models. Esses modelos foram exportados para poder reutilizar em outros arquivos.

CONTATO

```
3  const mongoose = require("mongoose");
4
5  const ContatoSchema = new mongoose.Schema({
6    nome: String,
7    email: String,
8    numero: String,
9    criadoEm: {
10     type: Date,
11     default: Date.now
12   }
13 });
14
15 module.exports = mongoose.model("Contato", ContatoSchema);
```

USUARIO

```
3  const mongoose = require("mongoose");
4
5  const UserSchema = new mongoose.Schema({
6    nome: String,
7    email: String,
8    usuario: String,
9    senha: String
10 });
11
12 module.exports = mongoose.model("User", UserSchema);
```

7.3 FETCH API

O método `fetch()` foi utilizado nos sistemas de login, registro e contato, assim enviando informações do navegador para o servidor em formato JSON, assim realizando a comunicação entre o front-end e o back-end sem recarregar a página.

```

103 fetch("/registro", {
104     method: "POST",
105     headers: {
106         "Content-Type": "application/json"
107     },
108     body: JSON.stringify({
109         nome: nome,
110         usuario: usuario,
111         email: email,
112         senha: senha
113     })
114 })

```

O servidor processa os dados e retorna uma resposta também em JSON:

```

115 .then(function (resposta) {
116     return resposta.json();

```

7.4 SISTEMA DE LOGIN E REGISTRO

Foi desenvolvido um sistema de autenticação simples contendo cadastro e login de usuários.

Entrar

Acesse sua conta

Usuário

Senha

Entrar

Não tem conta? [Criar conta](#)

Criar conta

Cadastre-se na Fazenda Urbana

Nome

Usuário

Email

Senha

Confirmar senha

Criar conta

[Já tenho conta](#)

Após logar ele muda para conectado e o nome do usuário.

Conectado: natal

No mongoDB aparece a lista de usuários cadastrado.

```
_id: ObjectId('6a137b0cd8a99740080c8e73')
nome : "lucas"
email : "lucasnatal8@gmail.com"
usuario : "natal"
senha : "123"
__v : 0
```

7.5 SISTEMA DE CONTATO

A página de contato também foi integrada com MongoDB, armazenando as informações enviadas pelo usuário.

FALE COM A FAZENDA URBANA

Preencha o formulário abaixo.

Digite seu nome:

Digite seu e-mail:

Digite seu numero:

Enviar

No MongoDB os pedidos de contato armazenado.

```
_id: ObjectId('6a137af3d8a99740080c8e72')
nome : "lucas"
email : "lucasnata18@gmail.com"
numero : "11960728414"
criadoEm : 2026-05-24T22:25:55.991+00:00
__v : 0
```

7.6 EJS E RENDERIZAÇÃO DE PÁGINAS

Foi utilizado como engine de visualização do projeto. Sua função foi permitir a renderização dinâmica das páginas HTML através do Express.js

```
21 app.set("view engine", "ejs");
22 app.set("views", path.join(__dirname, "views"));
23
```

8 ÁRVORE DA ESTRUTURA – N2

FazendaUrbana/

|----- database/

| |-----connection.js

|----- models/

| |----- user.js

| |----- contato.js

|-----public/

| |-----css/

| |-----global.css

| |-----js/

| |-----auth.js

| |-----contato.js

| |-----imagem.js

| |-----login.js

| |-----registro.js

| |-----script.js

| |-----assets/

| |-----img/

|-----views/

| |-----contato.ejs

| |-----etapas.ejs

| |----- impactos.ejs

| |-----index.ejs

```
|      |-----login.ejs
|      |-----parceiros.ejs
|----- server.js
|----- package-lock.json
|----- package.json
|----- node_modules/
```

9 CONCLUSÃO – N2

O desenvolvimento da segunda etapa do projeto Fazenda Urbana possibilitou a evolução do sistema de uma aplicação estática para uma aplicação web full stack funcional. Durante esta etapa, foram implementados recursos de back-end, integração com banco de dados e comunicação dinâmica entre cliente e servidor.

10 GITHUB

<https://github.com/ynatalz/Fazenda-Urbana>